

TweetUS Airline(Spark)

Periodo di Riferimento: Febbraio 2015

Nazione Rilevazione Tweet: USA

Totale Tweet: 14.640

Avvio Sessione Spark

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("progetto").getOrCreate()
```

Costruzione Schema

```
from pyspark.sql.types import StructField,StringType,IntegerType,StructType

data_schema = [StructField("tweet_id",StringType(),True),
                StructField("airline_sentiment",StringType(),True),
                StructField("airline_sentiment_confidence",StringType(),True),
                StructField("negativereason",StringType(),True),
                StructField("negativereason_confidence",StringType(),True),
                StructField("airline",StringType(),True),
                StructField("airline_sentiment_gold",StringType(),True),
                StructField("name",StringType(),True),
                StructField("negativereason_gold",StringType(),True),
                StructField("retweet_count",IntegerType(),True),
                StructField("text",StringType(),True),
                StructField("tweet_coord",StringType(),True),
                StructField("tweet_created",StringType(),True),
                StructField("tweet_location",StringType(),True),
                StructField("user_timezone",StringType(),True)
                ]

final_struc = StructType(fields = data_schema)
```

Abbiamo definito lo schema(la struttura) relativo al dataset da leggere. Il campo "tweet_id" rappresenta il codice univoco di un dato tweet. Il codice identificativo è di tipo Stringa, mentre il campo "retweet_count", che rappresenta il numero delle volte in cui un dato tweet è stato ritwittato è di tipo Intero.

Lettura del dataset

```
data = spark.read.csv("tweet.csv", schema = final_struct, header = True, nanValue="", sep='\t', multiline=True)
```

Dimensioni Dataset

```
print((data.count(), len(data.columns)))  
  
(14640, 15)
```

Il dataset è composto da 14640 osservazioni. Ogni osservazione rappresenta un tweet. Per ogni singola osservazione abbiamo 15 variabili.

Variabili di maggiore importanza

Le variabili più 'importanti' del dataset e che utilizzeremo nelle query sono:

- 1) **airline_sentiment**(Positivo, Negativo e Neutrale): Ogni tweet è classificato nelle tre categorie precedentemente citate
- 2) **airline**: Rappresenta il nome della compagnia aerea. Ogni tweet è riferito ad una determinata compagnia aerea
- 3) **name**: Nome Utente
- 4) **retweet_count**: Il numero di volte in cui un tweet specifico è stato ritwittato.
- 5) **text**: Commento dell'utente
- 6) **negativereason**: Ragione Negativa (Esempio: Bad Flight)
- 7) **user_timezone**: Indica il fuso orario dell'utente

Analitica 1

Quanti tweet per ciascun utente ?

La variabile da prendere in considerazione è **name**, che identifica il nome di un utente.

Query 1

```
from pyspark.sql.functions import count, desc  
data.groupBy("name").agg(count("name").alias("Totale Tweet"))\  
.sort(desc("Totale Tweet")).show(10)
```

Output Query 1

name	Totale Tweet
JetBlueNews	63
kbosspotter	32
_mhertz	29
otisday	28
throthra	27
rossj987	23
weezerandburnie	23
GREATNESSEOA	22
Meeestarcake	22
scoobydoo9749	21

only showing top 10 rows

Per comodità di visualizzazione ho modificato la query in modo tale da visualizzare in ordine decrescente(per numero di tweet scritti) i primi dieci utenti. `.sort(desc("Totale Tweet")).show(10)`

Commento Query 1

Con la clausola `groupBy` raggruppiamo per `name`(nome utente di Twitter), ed utilizziamo la funzione di aggregazione `count` per contare il numero di volte in cui compare il nome utente per cui stiamo raggruppando. Con il comando `sort` ordiniamo in modo decrescente il totale dei tweet rispetto all'autore. Con `show(10)` visualizziamo i primo 10 autori che hanno scritto più tweet.

Analitica 2

Quanti retweet per ciascuna compagnia aerea?

Le variabili da prendere in considerazione sono :

1) `retweet_count`

2) `airline`

`retweet_count` ci dice quante volte un determinato tweet è stato 'ritwittato', mentre `airline` indica il nome della

compagnia aerea.

Query 2

```
from pyspark.sql.functions import sum,desc,col

data.groupBy("airline").agg(sum("retweet_count").alias("Numero Retweet"))\
.filter(col("Numero Retweet") > 0)\
.sort(desc("Numero Retweet")).show()

# Otro Modo

data_filter = data.filter(data["retweet_count"] > 0)

data_filter.groupBy("airline").agg(sum("retweet_count").alias("Numero Retweet"))\
.sort(desc("Numero Retweet")).show()

# Otro Modo

data.filter(col("retweet_count") > 0).groupBy("airline").agg(sum("retweet_count").alias("Numero Retweet"))\
.sort(desc("Numero Retweet")).show()
```

Output Query 2

airline	Numero Retweet
United	421
Delta	252
US Airways	249
Southwest	145
American	117
Virgin America	26

airline	Numero Retweet
United	421
Delta	252
US Airways	249
Southwest	145
American	117
Virgin America	26

airline	Numero Retweet
United	421
Delta	252
US Airways	249
Southwest	145
American	117
Virgin America	26

Commento Query 2

1 Modo

Con la clausola `groupBy` raggruppiamo per `airline` (nome della compagnia aerea), ed utilizziamo la funzione di aggregazione `sum` per sommare il numero totale di `retweet` che si riferiscono alla compagnia aerea per cui stiamo raggruppando. Successivamente con la clausola `filter`, prendiamo in considerazione solo quelle compagnie aeree in cui la somma dei `retweet` è maggiore di 0. Con la clausola `sort` ordiniamo in ordine decrescente il numero totale di `retweet`.

2 Modo

Con la clausola `filter`, prendiamo in considerazione quelle osservazioni in cui il campo `retweet_count` è maggiore di 0. Successivamente andiamo a raggruppare per `airline` (nome compagnia aerea), non sul dataset "originale" ovvero `data`, ma su un subset del dataset originale ovvero `data_filter` che conterrà quelle osservazioni in cui il campo `retweet_count` è maggiore di 0. Successivamente utilizziamo la funzione di aggregazione `sum` per sommare il numero totale di `retweet` che si riferiscono alla compagnia aerea per cui stiamo raggruppando. Con la clausola `sort` ordiniamo in ordine decrescente il numero totale di `retweet`.

Analitica 3

TagCloud dei Termini

La variabile da utilizzare è text

Query 3

```
## Query 3

## Tag Cloud dei termini

from wordcloud import WordCloud, STOPWORDS

import matplotlib.pyplot as plt

text = data.select("text").collect()
stopwords = set(STOPWORDS)
text= list(text)
comments=''
for i in range(len(text)-1):
    comment=str(text[i] ["text"])
    comments+=comment+""

wordcloud=WordCloud(width=800,height=200, background_color="white", stopwords = stopwords).generate(comments)
plt.figure(figsize=(8,8),facecolor=None)
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```

Output Query 3



Commento Query 3

In questa query sono state importate tutte le librerie necessarie per fare la TagCloud dei Termini.

Con la clausola `select` abbiamo selezionato il campo di interesse ovvero `text(commento del utente)`.

set(STOPWORDS) restituisce invece (Una piccola parte):

'they're',
'they've',
'this',
'those',
'through',
'to',
'too',
'under',
'until',
'up',
'very',
'was',
'wasn't',
'we',
'we'd',
'we'll',
'we're',
'we've',
'were',

Se all'interno di una frase abbiamo le seguenti 'parole', esse vengono 'scartate' in quanto non rappresentano dei termini ma elementi che collegano i termini.

Possiamo aggiungere elementi alla lista STOPWORDS mediante il comando: `STOPWORDS.add('added')`.

Il comando `list(text)` crea una lista in cui sono presenti tutti i commenti degli utenti.

```
[Row(text='@VirginAmerica what @dhepburn said. '),
 Row(text="@VirginAmerica plus you've added commercials to the experience... tacky."),
```

Con i comandi:

```
for i in range(len(text)-1):  
    comment=str(text[i] ["text"])  
    comments+=comment+""
```

In output abbiamo:

```
@VirginAmerica What @dhepburn said.@VirginAmerica plus you've added commercials to the experience... tacky.
```

Con i comandi successivi andiamo a creare la TagCloud dei termini.

Analitica 4

Quale è la compagnia con il maggior numero di tweet con sentiment negativa?

Le variabili da prendere in considerazione sono:

1)airline

2) airline_sentiment

airline identifica il nome della compagnia aerea, mentre airline_sentiment identifica il sentiment degli utenti di twitter nei confronti di una compagnia aerea.

Query 4

```
from pyspark.sql.functions import count,desc  
data_filter = data.filter(data["airline_sentiment"] == "negative")  
data_filter.groupBy("airline").agg(count("airline").alias("Totale Tweet Negativi"))\  
.sort(desc("Totale Tweet Negativi")).show(1)  
  
# Altro Modo  
from pyspark.sql.functions import count, desc,col  
data.filter(col("airline_sentiment")== "negative").groupBy("airline").agg(count("airline").alias("Totale Tweet Negativi"))\  
.sort(desc("Totale Tweet Negativi")).show(1)
```

Output Query 4


```

+-----+-----+
|airline|Totale Tweet Negativi|
+-----+-----+
| United|                2633|
+-----+-----+

```

only showing top 1 row

```

+-----+-----+
|airline|Totale Tweet Negativi|
+-----+-----+
| United|                2633|
+-----+-----+

```

only showing top 1 row

Commento Query 4

1 Modo

Con la clausola filter, prendiamo in considerazione quelle osservazioni in cui il campo airline_sentiment è negativo. Successivamente andiamo a raggruppare per airline(nome compagnia aerea), non sul dataset "originale" ovvero data, ma su un subset del dataset originale ovvero data_filter che conterrà quelle osservazioni in cui il campo airline_sentiment è negativo. Successivamente utilizziamo la funzione di aggregazione count per contare il numero di tweet negativi relativi alla compagnia aerea per cui stiamo raggruppando. Con la clausola sort ordiniamo in ordine decrescente il numero totale di tweet negativi. Con show(1), in output avremo la compagnia aerea che ha ricevuto più tweet negativi.

2 Modo

Con la clausola filter, prendiamo in considerazione quelle osservazioni in cui il campo airline_sentiment è negativo. Successivamente andiamo a raggruppare per airline(nome compagnia aerea) ed utilizziamo la funzione di aggregazione count per contare il numero di tweet negativi relativi alla compagnia aerea per cui stiamo raggruppando. Con la clausola sort ordiniamo in ordine decrescente il numero totale di tweet negativi. Con show(1), in output avremo la compagnia aerea che ha ricevuto più tweet negativi.

Analitica 5

Distribuzione sentiment per ciascuna compagnia aerea

Le variabili da prendere in considerazione sono:

1) airline

2) airline_sentiment

airline identifica il nome della compagnia aerea mentre airline_sentiment identifica il sentiment degli utenti di twitter nei confronti di una compagnia aerea.

Query 5

```
from pyspark.sql.functions import count,desc,col

data.groupBy("airline","airline_sentiment") \
    .agg(count("airline_sentiment").alias("Conteggio"))\
    .sort(desc("airline")).show()
```

Output Query 5

airline	airline_sentiment	Conteggio
Virgin America	negative	181
Virgin America	neutral	171
Virgin America	positive	152
United	negative	2633
United	neutral	697
United	positive	492
US Airways	negative	2263
US Airways	neutral	381
US Airways	positive	269
Southwest	negative	1186
Southwest	positive	570
Southwest	neutral	664
Delta	positive	544
Delta	neutral	723
Delta	negative	955
American	positive	336
American	negative	1960
American	neutral	463

Commento Query 5

Con la clausola `groupBy` raggruppiamo per `airline` e `airline_sentiment`. Successivamente utilizziamo la funzione di aggregazione `count` per contare il numero di tweet negativi, positivi e neutrali relativi alla compagnia aerea per cui stiamo raggruppando. Con la clausola `sort` ordiniamo in ordine decrescente il campo `airline`.

Analitica 6

Distribuzione del sentiment per ciascuna motivazione

La variabile da prendere in considerazione è `negativereason`

Query 6

```
from pyspark.sql.functions import count, desc, col

data_filter = data.filter(data["airline_sentiment"] == "negative")

data_filter.groupBy("negativereason") \
    .agg(count("negativereason").alias("Totale Ragioni Negative")) \
    .sort(desc("Totale Ragioni Negative")).show()

# Altro Modo

data.filter(col("airline_sentiment") == "negative").groupBy("negativereason") \
    .agg(count("negativereason").alias("Totale Ragioni Negative")) \
    .sort(desc("Totale Ragioni Negative")).show()
```

Output Query 6

negativereason	Totale Ragioni Negative
Customer Service ...	2910
Late Flight	1665
Can't Tell	1190
Cancelled Flight	847
Lost Luggage	724
Bad Flight	580
Flight Booking Pr...	529
Flight Attendant ...	481
longlines	178
Damaged Luggage	74

negativereason	Totale Ragioni Negative
Customer Service ...	2910
Late Flight	1665
Can't Tell	1190
Cancelled Flight	847
Lost Luggage	724
Bad Flight	580
Flight Booking Pr...	529
Flight Attendant ...	481
longlines	178
Damaged Luggage	74

Commento Query 6

1 Modo

Con la clausola filter, prendiamo in considerazione quelle osservazioni in cui il campo airline_sentiment è negativo. Successivamente andiamo a raggruppare non sul dataset "originale" ovvero data, ma su un subset del dataset originale ovvero data_filter che conterrà quelle osservazioni in cui il campo airline_sentiment è negativo. Utilizziamo la funzione di aggregazione count per contare quante volte si presenta la ragione negativa per cui stiamo raggruppando. Con la clausola sort ordiniamo in ordine decrescente le ragioni negative.

2 Modo

Con la clausola filter, prendiamo in considerazione quelle osservazioni in cui il campo airline_sentiment è negativo. Successivamente andiamo a raggruppare per negativereason(ragione negativa) ed utilizziamo la funzione di aggregazione count per contare quante volte si presenta la ragione negativa per cui stiamo raggruppando. Con la clausola sort ordiniamo in ordine decrescente le ragioni negative.

Analitica 7

Autori, in ordine alfabetico, che hanno ricevuto più di 10 retweet

Le variabili da prendere in considerazione sono :

1)name

2)retweet_count

name identifica l'autore del tweet, mentre retweet_count ci dice quante volte un determinato tweet è stato 'ritwittato'.

Query 7

```
from pyspark.sql.functions import sum,asc,col
data.groupBy("name").agg(sum("retweet_count").alias("Numero Retweet"))\
.filter(col("Numero Retweet") > 10)\
.sort(asc("name")).show()
```

Output Query 7

name	Numero Retweet
AbeFroman	18
JasonShaw2	31
NickTypesWords	20
No_Cut_Card	15
OBJ_3	80
OlivierGachot	11
TatianaKing	31
TheBuie	22
airliners2	28
davidgoodson71	13
seattlefordguy	22

Commento Query 7

Con la clausola `groupBy` raggruppiamo per `name` (nome utente di Twitter), ed utilizziamo la funzione di aggregazione `sum` per sommare il numero di retweet relativi al nome utente per cui stiamo raggruppando. Con la clausola `filter`, prendiamo in considerazione gli utenti che hanno ricevuto più di 10 retweet. Con il comando `sort` ordiniamo in modo crescente `name`.

Analitica 8

Data in input una compagnia aerea, mostrare la TagCloud dei termini utilizzati in caso di sentiment negativa.

Query 8

```
## Query 8

## Data in input una compagnia aerea,
## mostrare la tagCloud dei termini utilizzati in caso di sentiment negativa

from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt

nome_compagnia = input("Inserisci il nome di una compagnia aerea:\t")

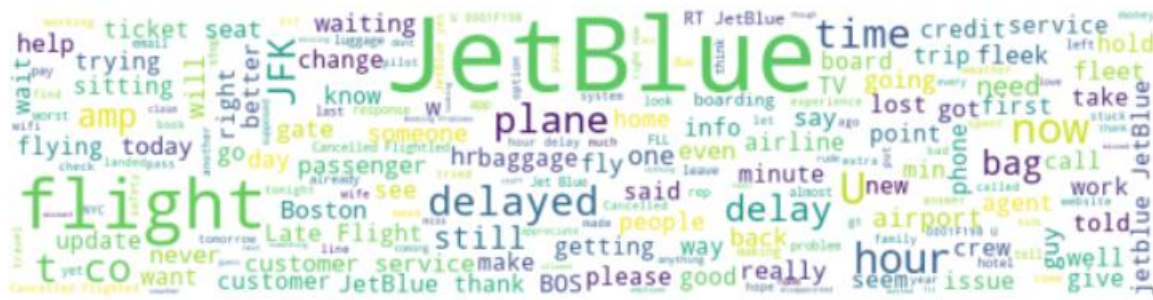
data_filter = data.filter((data["airline"] == nome_compagnia) & (data["airline_sentiment"] == "negative"))

text = data_filter.select("text").collect()
stopwords = set(STOPWORDS)
text = list(text)
comments = ''
for i in range(len(text)-1):
    comment = str(text[i] ["text"])
    comments += comment + ""

wordcloud = WordCloud(width=800, height=200, background_color="white").generate(comments)
plt.figure(figsize=(8,8), facecolor=None)
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```

Output Query 8

Delta



Commento Query 8

Con la clausola `filter` prendiamo in considerazione solo quelle osservazioni in cui i campi `airline` e `airline_sentiment` sono rispettivamente uguali all'input dato e a `negative`. Successivamente andiamo a selezionare il campo `text` non sul dataset "originale" ovvero `data`, ma su un subset del dataset originale ovvero `data_filter`. I restanti comandi sono uguali ai comandi già spiegati della query numero 3.

Analitica 9

Data in input una compagnia aerea, mostrare la TagCloud dei termini utilizzati in caso di sentiment positiva.

Query 9

TweetUS Airline(Spark ML)

Avvio Sessione Spark

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("spark_ml").getOrCreate()
```

Costruzione Schema

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("spark_ml").getOrCreate()

from pyspark.sql.types import StructField,StringType,IntegerType,StructType

data_schema = [StructField("tweet_id",StringType(),True),
                StructField("airline_sentiment",StringType(),True),
                StructField("airline_sentiment_confidence",IntegerType(),True),
                StructField("negativereason",StringType(),True),
                StructField("negativereason_confidence",StringType(),True),
                StructField("airline",StringType(),True),
                StructField("airline_sentiment_gold",StringType(),True),
                StructField("name",StringType(),True),
                StructField("negativereason_gold",StringType(),True),
                StructField("retweet_count",IntegerType(),True),
                StructField("text",StringType(),True),
                StructField("tweet_coord",StringType(),True),
                StructField("tweet_created",StringType(),True),
                StructField("tweet_location",StringType(),True),
                StructField("user_timezone",StringType(),True)
                ]

final_struc = StructType(fields = data_schema)
```

Lettura del dataset

```
data = spark.read.csv("tweet.csv", schema = final_struc, header = True, nanValue="", sep='\t', multiline=True)
```

Rimuovo Na

```
data = data.na.drop()
```

Dimensioni Dataset

```
print((data.count(), len(data.columns)))  
(10445, 19)
```

Analitica 2

Considerando le feature del dataset realizzare un modello predittivo in grado di stimare il sentiment per la compagnia (positivo/negativo).

Feature Transformers

Tecniche utilizzate

- Tokenizer
- StopWordsRemover

Tokenizer

La tokenizzazione è il processo mediante il quale un testo (come una frase) viene scomposto in termini individuali (di solito parole).

Esempio



StopWordsRemover

StopWordsRemover prende come input una sequenza di stringhe (ad esempio l'output di un Tokenizer) e rilascia tutte le parole di stop dalle sequenze di input. L'elenco delle parole non significative è specificato dal parametro stopWords. Le parole chiave predefinite per alcune lingue sono accessibili chiamando StopWordsRemover.loadDefaultStopWords (lingua). Un parametro booleano caseSensitive indica se le corrispondenze devono fare distinzione tra maiuscole e minuscole.

Esempio

Sample text with Stop Words	Without Stop Words
GeeksforGeeks – A Computer Science Portal for Geeks	GeeksforGeeks , Computer Science, Portal ,Geeks
Can listening be exhausting?	Listening, Exhausting
I like reading, so I read	Like, Reading, read

Codice Tokenizer + StopWordsRemover

```
from pyspark.ml.feature import Tokenizer, StopWordsRemover

tokenizer_text = Tokenizer(inputCol = "text", outputCol = "token_text")

stopremove_text = StopWordsRemover(inputCol = "token_text", outputCol = "stop_tokens_text")

tokenizer_negativereason = Tokenizer(inputCol = "negativereason", outputCol = "token_negative_reason")

stopremove_negativereason = StopWordsRemover(inputCol = "token_negative_reason", outputCol = "stop_tokens_negative_reason")
```

In questo caso l'input del Tokenizer è text. Text rappresenta il commento del utente di Twitter. token_text rappresenta l'output del tokenizer applicato sul campo text. L'input dello StopWordsRemover è token_text ovvero l'output del Tokenizer. L'output dello StopWordsRemover è stop_tokens. Discorso analogo per negativereason che rappresenta la ragione negativa espressa dal utente di twitter.

Feature Extractors

Tecniche utilizzate

- CountVectorizer
- TF-IDF

CountVectorizer(Usato in alternativa a HashingTF)

CountVectorizer può essere utilizzato per generare i vettori di frequenza del termine. CountVectorizer prende in input le parole e restituisce un conteggio.

Esempio

Data = ['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog']



	The	quick	brown	fox	jumps	over	lazy	dog
Data	2	1	1	1	1	1	1	1

TF-IDF

L'algoritmo TF-IDF associa maggiore importanza ai termini meno frequenti all'interno del documento ma più rilevanti per la ricerca. Le parole comuni sono molto frequenti ma, essendo anche presenti in tutti i documenti, hanno una ridotta importanza nel processo di selezione delle risorse. Un'alternativa a HashingTF è CountVectorizer, utilizzato in precedenza che genera i vettori di frequenza del termine.

IDF = $\log N / nk$. Se il termine K è presente in tre documenti ($nk=3$) su cento documenti ($N=100$), allora il valore di IDF è pari a 1.5. Se il termine K è presente in venticinque documenti ($nk=25$) su cento documenti ($N=100$), allora il valore di IDF è pari a 0.60. Questo segue la logica precedente, ovvero i termini meno frequenti hanno un peso maggiore.

Codice CountVectorizer + IDF

```
count_vec_text = CountVectorizer(inputCol = "stop_tokens_text", outputCol = "c_vec_text")
idf_text = IDF(inputCol = "c_vec_text", outputCol = "tf_idf_text")
count_vec_negativereason = CountVectorizer(inputCol = "stop_tokens_negative_reason", outputCol = "negative_reason_vec")
idf_negativereason = IDF(inputCol = "negative_reason_vec", outputCol = "tf_idf_negative_reason")
```

L'input del CountVectorizer è l'output dello StopWordsRemover ovvero stop_tokens_text. L'output di CountVectorizer è c_vec_text ovvero il vettore di frequenza dei termini che riguarda il campo text. L'input

del IDF è il vettore di frequenza dei termini `c_vec_text`. L'output del IDF è il `tf_idf_text`. Discorso analogo per `negativereason` che rappresenta la ragione negativa espressa dal utente di twitter.

Con le tecniche di **Feature Transformers** e di **Feature Extractors** abbiamo trasformato e successivamente estratto informazioni utili a partire da commenti generati dagli utenti di twitter.

Classi da prevedere

```
data.select('airline_sentiment').distinct().collect()
```

```
[Row(airline_sentiment='positive'),  
 Row(airline_sentiment='neutral'),  
 Row(airline_sentiment='negative')]
```

Le classi da prevedere sono: positive, neutral e negative. Siccome la classe del nostro dataset è una parola, utilizziamo lo `StringIndexer` che è una tecnica di **Feature Transformers**. `StringIndexer` associa ad ogni classe un numero. Questo numero rappresenta la classe di appartenenza.

Codice StringIndexer

```
has_sentiment_to_num = StringIndexer(inputCol = "airline_sentiment", outputCol = "sentimentIndex")  
data = has_sentiment_to_num.fit(data).transform(data)  
  
has_airline_to_num = StringIndexer(inputCol = "airline", outputCol = "airlineIndex")  
data = has_airline_to_num.fit(data).transform(data)
```

In questo caso abbiamo applicato lo `stringIndexer` sia al campo `airline_sentiment` sia al campo `airline`.

Con `has_sentiment_to_num.fit(data).transform(data)` andiamo ad applicare lo `stringIndexer` al campo `airline_sentiment` e trasformiamo il nostro dataset, ovvero aggiungiamo al nostro dataset la colonna `has_sentiment_to_num`. Discorso analogo per `has_airline_to_num`.

OneHotEncoder

Un'altra tecnica di Feature Transformers è il OneHotEncoder. Il OneHotEncoder converte una variabile categoriale in una rappresentazione numerica non ordinale da utilizzare negli algoritmi di apprendimento automatico.

Esempio

	Travel_Class_2	Travel_Class_3
Passenger 1	0	0
Passenger 2	1	0
Passenger 3	0	1
Passenger 4	0	0

Codice OneHotEncoder

```
onehotencoder_sentiment_vector = OneHotEncoder(inputCol="sentimentIndex", outputCol="sentiment_vec")
data = onehotencoder_sentiment_vector.fit(data).transform(data)

onehotencoder_airline_vector = OneHotEncoder(inputCol="airlineIndex", outputCol="airline_vec")
data = onehotencoder_airline_vector.fit(data).transform(data)
```

In questo caso abbiamo applicato il OneHotEncoder sia a sentimentIndex sia a airlineIndex che rappresentano l'output dello stringIndexer. Come prima abbiamo applicato il OneHotEncoder ai campi interessati e abbiamo aggiunto le due nuove colonne al dataset. Una colonna è relativa al onehotencoding di sentimentIndex, un'altra colonna è relativa

Colonne dataset aggiornato

Il modello di classificazione utilizzato è il LogisticRegression. labelCol indica la variabile dipendente ovvero sentimentIndex, mentre le variabili indipendenti da dare in input al nostro modello di classificazione sono le features indicate al passaggio precedente.(Vector Assembler). prediction è la colonna che contiene le previsioni.

Codice Pipeline

La pipeline concatenata più trasformatori e non solo. All'interno della pipeline abbiamo:

- 1) L'output del Tokenizer riguardante il campo text
- 2) L'output relativo allo StopWordsRemover riguardante il campo text
- 3) L'output di CountVectorizer riguardante il campo text
- 4) L'output del IDF riguardante il campo text
- 5) L'output del Tokenizer riguardante il campo negativereason
- 6) L'output relativo allo StopWordsRemover riguardante il campo negativereason
- 7) L'output di CountVectorizer riguardante il campo negativereason
- 8) L'output del IDF riguardante il campo negativereason
- 9) clean_up rappresenta il vettore delle features assemblate nel VectorAssembler

```
from pyspark.ml import Pipeline

data_prep_pipe = Pipeline(stages=[tokenizer_text, stopremove_text, count_vec_text, idf_text,
                                  tokenizer_negativereason, stopremove_negativereason,
                                  count_vec_negativereason, idf_negativereason, clean_up])

cleaner = data_prep_pipe.fit(data)
```

```
clean_data = cleaner.transform(data)
```

```
clean_data.head()
```

```
Row(tweet_id='570306133677760512', airline_sentiment='neutral', airline_sentiment_confidence=1, negativereason='NA', negativereason_confidence='NA', airline='Virgin America', airline_sentiment_gold='NA', name='cairdin', negativereason_gold='NA', retweet_count=0, text='@VirginAmerica What @dhepburn said.', tweet_coord='NA', tweet_created='2015-02-24 11:35:52 -0800', tweet_location='NA', user_timezone='Eastern Time (US & Canada)', sentimentIndex=1.0, sentiment_vec=SparseVector(2, {1: 1.0}), airlineIndex=5.0, airline_vec=SparseVector(5, {}), token_text=['@virginamerica', 'what', '@dhepburn', 'said.'], stop_tokens=['@virginamerica', '@dhepburn', 'said.'], c_vec=SparseVector(21246, {31: 1.0, 4268: 1.0, 12682: 1.0}), tf_idf=SparseVector(21246, {31: 3.4794, 4268: 7.8677, 12682: 8.5608}), token_negative_reason=['na'], negative_reason_vec=SparseVector(18, {1: 1.0}), tf_idf_negative_reason=SparseVector(18, {1: 1.2265}), features=SparseVector(21271, {31: 3.4794, 4268: 7.8677, 12682: 8.5608, 21247: 1.0, 21254: 1.2265}))
```


Colonne dataset prima della pipeline

data.columns

```
['tweet_id',  
'airline_sentiment',  
'airline_sentiment_confidence',  
'negativereason',  
'negativereason_confidence',  
'airline',  
'airline_sentiment_gold',  
'name',  
'negativereason_gold',  
'retweet_count',  
'text',  
'tweet_coord',  
'tweet_created',  
'tweet_location',  
'user_timezone',  
'sentimentIndex',  
'sentiment_vec',  
'airlineIndex',  
'airline_vec']
```

Colonne dataset dopo pipeline

clean_data.columns

```
['tweet_id',  
'airline_sentiment',  
'airline_sentiment_confidence',  
'negativereason',  
'negativereason_confidence',  
'airline',  
'airline_sentiment_gold',  
'name',  
'negativereason_gold',  
'retweet_count',  
'text',  
'tweet_coord',  
'tweet_created',  
'tweet_location',  
'user_timezone',  
'sentimentIndex',  
'sentiment_vec',  
'airlineIndex',  
'airline_vec',  
'token_text',  
'stop_tokens',  
'c_vec',  
'tf_idf_text',  
'token_negative_reason',  
'stop_tokens_negative_reason',  
'negative_reason_vec',  
'tf_idf_negative_reason',  
'features']
```

Codice divisione Train e Test Set

```
(training,testing) = clean_data.randomSplit([0.7,0.3], seed = 1)  
  
print("Number of training set rows: %d" % training.count())  
print("Number of test set rows: %d" % testing.count())
```

```
Number of training set rows: 7349  
Number of test set rows: 3096
```

Abbiamo fissato il seed in modo tale da evitare l'effetto casuale dovuto al campionamento. Lo stesso train e lo stesso test set saranno utilizzati per addestrare modelli differenti.

Valutazione del Modello

```

multinomial_logistic = mnlogit.fit(training)

predictions_test = multinomial_logistic.transform(testing)

sentiment_true = predictions_test.select(['sentimentIndex']).collect()
sentiment_pred = predictions_test.select(['prediction']).collect()

from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(sentiment_true, sentiment_pred))

```

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	2174
1.0	0.99	1.00	0.99	463
2.0	1.00	0.97	0.98	459
accuracy			0.99	3096
macro avg	0.99	0.99	0.99	3096
weighted avg	0.99	0.99	0.99	3096

La matrice di confusione si riferisce al test set. Nel test set abbiamo in totale 3096 osservazioni. 0 rappresenta la classe dei negativi, 1 rappresenta la classe dei neutrali mentre 2 rappresenta la classe dei positivi. Quindi nel test set abbiamo 2174 osservazioni che rappresentano la classe dei tweet negativi, 463 osservazioni che rappresentano la classe dei tweet neutrali e infine 459 osservazioni che rappresentano la classe dei tweet positivi.

Precision = (TRUE POSITIVE) / (TRUE POSITIVE + FALSE POSITIVE)

Recall = (TRUE POSITIVE) / (TRUE POSITIVE + FALSE NEGATIVE)

Codice per estrarre le prime due previsioni per ogni classe

```

predictions = multinomial_logistic.transform(testing)

predictions.filter(predictions['prediction'] == 0) \
  .select("airline_sentiment", "sentimentIndex", "negativereason", "probability", "prediction") \
  .orderBy("probability", ascending=False) \
  .show(n = 3, truncate = 30)

predictions.filter(predictions['prediction'] == 1) \
  .select("airline_sentiment", "sentimentIndex", "negativereason", "probability", "prediction") \
  .orderBy("probability", ascending=False) \
  .show(n = 2, truncate = 30)

predictions.filter(predictions['prediction'] == 2) \
  .select("airline_sentiment", "sentimentIndex", "negativereason", "probability", "prediction") \
  .orderBy("probability", ascending=False) \
  .show(n = 2, truncate = 30)

```

Output

airline_sentiment	sentimentIndex	negativereason	probability	prediction
negative	0.0	Late Flight	[1.0, 6.772537258549145E-17, ...]	0.0
negative	0.0	Can't Tell	[1.0, 5.641948384346585E-17, ...]	0.0
negative	0.0	Customer Service Issue	[1.0, 3.8227880230791544E-17, ...]	0.0

only showing top 3 rows

airline_sentiment	sentimentIndex	negativereason	probability	prediction
positive	2.0	NA	[0.3899500170102226, 0.49245..., ...]	1.0
positive	2.0	NA	[0.14444177713786413, 0.6625..., ...]	1.0

only showing top 2 rows

airline_sentiment	sentimentIndex	negativereason	probability	prediction
positive	2.0	NA	[0.3376711510226177, 0.05071..., ...]	2.0
positive	2.0	NA	[0.2367475484778395, 0.01134..., ...]	2.0

only showing top 2 rows

Codice Accuratezza

```

from pyspark.ml.evaluation import MulticlassClassificationEvaluator

evaluator = MulticlassClassificationEvaluator(predictionCol="prediction",labelCol = 'sentimentIndex')

print("Accuracy:\t",evaluator.evaluate(predictions))

print("MisClassificationRate:\t" ,1 - evaluator.evaluate(predictions))

```

```

Accuracy:          0.9948000302474553
MisClassificationRate:  0.005199969752544664

```

Analitica 1

Considerando le feature del dataset realizzare un modello predittivo in grado di stimare il numero di re-tweet di un tweet.

Avvio Sessione Spark

```

from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("stima_retweet").getOrCreate()

```

Costruzione Schema

```

data_schema = [StructField("tweet_id",StringType(),True),
                StructField("airline_sentiment",StringType(),True),
                StructField("airline_sentiment_confidence",IntegerType(),True),
                StructField("negativereason",StringType(),True),
                StructField("negativereason_confidence",StringType(),True),
                StructField("airline",StringType(),True),
                StructField("airline_sentiment_gold",StringType(),True),
                StructField("name",StringType(),True),
                StructField("negativereason_gold",StringType(),True),
                StructField("retweet_count",IntegerType(),True),
                StructField("text",StringType(),True),
                StructField("tweet_coord",StringType(),True),
                StructField("tweet_created",StringType(),True),
                StructField("tweet_location",StringType(),True),
                StructField("user_timezone",StringType(),True)
                ]

final_struc = StructType(fields = data_schema)

```

Lettura del dataset

```
data = spark.read.csv("tweet.csv", schema = final_struc, header = True, nanValue="", sep='\t', multiline=True)
```

Rimuovo Na

```
data = data.na.drop()
```

Le tecniche di feature Feature Extractors e di Feature Transformers sono le stesse applicate al punto precedente (Analitica 2).

Codice Feature Extractors e di Feature Transformers

```
## Feature Transformations
from pyspark.ml.feature import Tokenizer, StopWordsRemover, CountVectorizer, IDF, StringIndexer, OneHotEncoder

tokenizer_text = Tokenizer(inputCol = "text", outputCol = "token_text")
stopremove_text = StopWordsRemover(inputCol = "token_text", outputCol = "stop_tokens")
count_vec_text = CountVectorizer(inputCol = "stop_tokens", outputCol = "c_vec")
idf_text = IDF(inputCol = "c_vec", outputCol = "tf_idf_text")

tokenizer_negativereason = Tokenizer(inputCol = "negativereason", outputCol = "token_negative_reason")
stopremove_negativereason = StopWordsRemover(inputCol = "token_negative_reason", outputCol = "stop_tokens_negative_reason")
count_vec_negativereason = CountVectorizer(inputCol = "stop_tokens_negative_reason", outputCol = "negative_reason_vec")
idf_negativereason = IDF(inputCol = "negative_reason_vec", outputCol = "tf_idf_negative_reason")

has_sentiment_to_num = StringIndexer(inputCol = "airline_sentiment", outputCol = "sentimentIndex")
data = has_sentiment_to_num.fit(data).transform(data)

onehotencoder_sentiment_vector = OneHotEncoder(inputCol="sentimentIndex", outputCol="sentiment_vec")
data = onehotencoder_sentiment_vector.fit(data).transform(data)

has_airline_to_num = StringIndexer(inputCol = "airline", outputCol = "airlineIndex")
data = has_airline_to_num.fit(data).transform(data)

onehotencoder_airline_vector = OneHotEncoder(inputCol="airlineIndex", outputCol="airline_vec")
data = onehotencoder_airline_vector.fit(data).transform(data)
```

Codice VectorAssembler

```

from pyspark.ml.feature import VectorAssembler
from pyspark.ml.linalg import Vector

clean_up = VectorAssembler(inputCols = ['tf_idf_text', 'sentiment_vec', 'airline_sentiment_confidence',
                                         'tf_idf_negative_reason'], outputCol = 'features')

```

Codice modello di regressione

```

from pyspark.ml.regression import LinearRegression

linear_regression = LinearRegression(featuresCol = 'features',
                                     labelCol = 'retweet_count',
                                     predictionCol = 'prediction')

```

Il modello di machine learning utilizzato è la LinearRegression. labelCol indica la variabile dipendente ovvero retweet_count, mentre le variabili indipendenti da dare in input al nostro modello di classificazione sono le features indicate al passaggio precedente.(Vector Assembler). prediction è la colonna che contiene le previsioni.

Codice Pipeline

La pipeline concatena più trasformatori e non solo. All'interno della pipeline abbiamo:

- 1) L'output del Tokenizer riguardante il campo text
- 2) L'output relativo allo StopWordsRemover riguardante il campo text
- 3) L'output di CountVectorizer riguardante il campo text
- 4) L'output del IDF riguardante il campo text
- 5) L'output del Tokenizer riguardante il campo negativereason
- 6) L'output relativo allo StopWordsRemover riguardante il campo negativereason
- 7) L'output di CountVectorizer riguardante il campo negativereason
- 8) L'output del IDF riguardante il campo negativereason
- 9) clean_up rappresenta il vettore delle features assemblate nel VectorAssembler

```

from pyspark.ml import Pipeline

data_prep_pipe = Pipeline(stages=[tokenizer_text,stopremove_text,count_vec_text,idf_text,
                                tokenizer_negativereason,stopremove_negativereason,
                                count_vec_negativereason,idf_negativereason,clean_up])

cleaner = data_prep_pipe.fit(data)

```

```
clean_data = cleaner.transform(data)
```

```
clean_data.head()
```

```

Row(tweet_id='570306133677760512', airline_sentiment='neutral', airline_sentiment_confidence=1, negativereason='NA', negativereason_confidence='NA', airline='Virgin America', airline_sentiment_gold='NA', name='cairdin', negativereason_gold='NA', retweet_count=0, text='@VirginAmerica What @dhepburn said.', tweet_coord='NA', tweet_created='2015-02-24 11:35:52 -0800', tweet_location='NA', user_timezone='Eastern Time (US & Canada)', sentimentIndex=1.0, sentiment_vec=SparseVector(2, {1: 1.0}), airlineIndex=5.0, airline_vec=SparseVector(5, {}), token_text=['@virginamerica', 'what', '@dhepburn', 'said.'], stop_tokens=['@virginamerica', '@dhepburn', 'said.'], c_vec=SparseVector(21246, {31: 1.0, 4268: 1.0, 12682: 1.0}), tf_idf=SparseVector(21246, {31: 3.4794, 4268: 7.8677, 12682: 8.5608}), token_negative_reason=['na'], negative_reason_vec=SparseVector(18, {1: 1.0}), tf_idf_negative_reason=SparseVector(18, {1: 1.2265}), features=SparseVector(21271, {31: 3.4794, 4268: 7.8677, 12682: 8.5608, 21247: 1.0, 21254: 1.2265}))

```

Codice divisione Train e Test Set

```

(training,testing) = clean_data.randomSplit([0.7,0.3], seed = 1)

print("Number of training set rows: %d" % training.count())
print("Number of test set rows: %d" % testing.count())

```

```

Number of training set rows: 7349
Number of test set rows: 3096

```

Valutazione del Modello

```

linear_regression_model = linear_regression.fit(training)

linear_regression_predictions = linear_regression_model.transform(testing)

from pyspark.ml.evaluation import RegressionEvaluator

linear_regression_evaluator = RegressionEvaluator(predictionCol="prediction",
                                                labelCol="retweet_count",metricName="rmse")

print("Root Mean Square Error (rmse) test data = %g" % linear_regression_evaluator.evaluate(linear_regression_predictions))

linear_regression_evaluator = RegressionEvaluator(predictionCol="prediction",
                                                labelCol="retweet_count",metricName="mae")

print("Mean Absolute Error(mae) test data = %g" % linear_regression_evaluator.evaluate(linear_regression_predictions))

```

Output

```
Root Mean Square Error (rmse) test data = 0.962666  
Mean Absolute Error(mae) test data = 0.341932
```

Per la valutazione del modello sono state prese in considerazione due metriche. La prima è il Root Mean Square Error(rmse) mentre la seconda è il Mean Absolute Error(mae).

RMSE = Somma delle differenze al quadrato tra il valore previsto e il valore reale. La somma viene divisa per la numerosità del campione. Infine, al risultato ottenuto applichiamo la radice quadrata.

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (Predicted_i - Actual_i)^2}{N}}$$

MAE = Somma della differenza in valore assoluto tra il valore reale e il valore previsto. La somma viene divisa per la numerosità del campione.

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j|$$

Task Bonus

Avvio Sessione Spark

```
from pyspark.sql import SparkSession  
spark = SparkSession.builder.appName("task_bonus").getOrCreate()
```

Costruzione Schema


```

from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("spark_ml").getOrCreate()

from pyspark.sql.types import StructField,StringType,IntegerType,StructType

data_schema = [StructField("tweet_id",StringType(),True),
                StructField("airline_sentiment",StringType(),True),
                StructField("airline_sentiment_confidence",IntegerType(),True),
                StructField("negativereason",StringType(),True),
                StructField("negativereason_confidence",StringType(),True),
                StructField("airline",StringType(),True),
                StructField("airline_sentiment_gold",StringType(),True),
                StructField("name",StringType(),True),
                StructField("negativereason_gold",StringType(),True),
                StructField("retweet_count",IntegerType(),True),
                StructField("text",StringType(),True),
                StructField("tweet_coord",StringType(),True),
                StructField("tweet_created",StringType(),True),
                StructField("tweet_location",StringType(),True),
                StructField("user_timezone",StringType(),True)
                ]

final_struc = StructType(fields = data_schema)

```

Lettura del dataset

```

data = spark.read.csv("tweet.csv", schema = final_struc, header = True, nanValue="", sep='\t', multiline=True)

```

Rimuovo Na

```

data = data.na.drop()

```

Le tecniche di Feature Extractors utilizzate sono le stesse utilizzate per l'analitica 1 e per l'analitica 2.

Feature Transformers

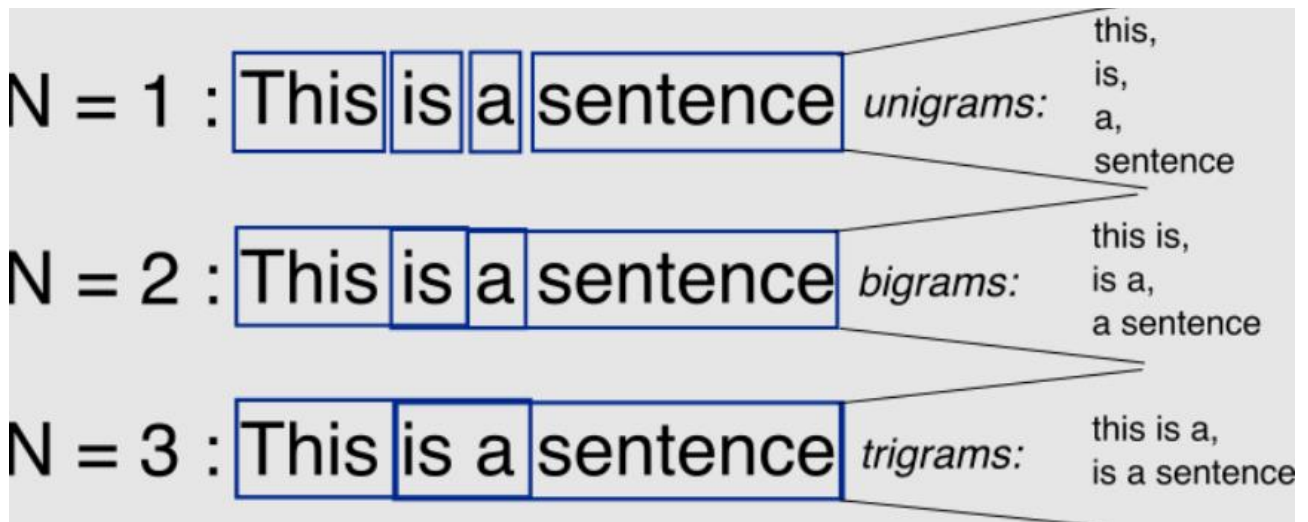
Tecniche utilizzate

- Tokenizer
- StopWordsRemover
- n-gram

Per il task bonus ho aggiunto una nuova tecnica di Feature Transformers ovvero **n-gram**.

n-gram

Esempio



Codice n-gram task bonus

```
tokenizer = Tokenizer(inputCol = "text", outputCol = "token_text")
data = tokenizer.transform(data)

from pyspark.ml.feature import NGram

ngram = NGram(n=2, inputCol="token_text", outputCol="ngrams")
data = ngram.transform(data)
```

L'input del n-gram è l'output della tokenizzazione ovvero token_text. Da token_text vogliamo ottenere degli n-grammi di parole adiacenti. Gli n-grammi presi in considerazione sono i bi-grammi, in quanto n = 2 (due parole adiacenti). La finestra che ci interessa è di dimensione due e quindi ci muoviamo di uno alla volta.

Esempio bi-gramma con n = 2



Codice Feature Transformers + Feature Extractors(Completo)

```
## Feature Transformations
from pyspark.ml.feature import Tokenizer, StopWordsRemover, CountVectorizer, IDF, StringIndexer, OneHotEncoder

has_sentiment_to_num = StringIndexer(inputCol = "airline_sentiment", outputCol = "sentimentIndex")
data = has_sentiment_to_num.fit(data).transform(data)

onehotencoder_sentiment_vector = OneHotEncoder(inputCol="sentimentIndex", outputCol="sentiment_vec")
data = onehotencoder_sentiment_vector.fit(data).transform(data)

has_airline_to_num = StringIndexer(inputCol = "airline", outputCol = "airlineIndex")
data = has_airline_to_num.fit(data).transform(data)

onehotencoder_airline_vector = OneHotEncoder(inputCol="airlineIndex", outputCol="airline_vec")
data = onehotencoder_airline_vector.fit(data).transform(data)

tokenizer = Tokenizer(inputCol = "text", outputCol = "token_text")

data = tokenizer.transform(data)

from pyspark.ml.feature import NGram
ngram = NGram(n=2, inputCol="token_text", outputCol="ngrams")
data = ngram.transform(data)

count_vec = CountVectorizer(inputCol = "ngrams", outputCol = "ngrams_c_vec")

idf = IDF(inputCol = "ngrams_c_vec", outputCol = "ngrams_tf_idf")
```

Codice VectorAssembler

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.linalg import Vector

clean_up = VectorAssembler(inputCols = ['ngrams_tf_idf', 'retweet_count', 'airline_sentiment_confidence'], outputCol = 'features')
```

Codice Pipeline

```
from pyspark.ml import Pipeline
data_prep_pipe = Pipeline(stages=[count_vec,idf, clean_up])
cleaner = data_prep_pipe.fit(data)

clean_data = cleaner.transform(data)
```

La pipeline concatena più trasformatori e non solo. All'interno della pipeline abbiamo inserito:

- 1) L'output di CountVectorizer riguardante il campo text
- 2) L'output del IDF riguardante il campo text
- 3) clean_up rappresenta il vettore delle features assemblate nel VectorAssembler

Molti dei trasformatori sono stati già applicati al dataset come visto nel codice

([Codice Feature Transformers + Feature Extractors\(Completo\)](#)), tuttavia questo rappresenta una scelta. Potevamo anche concatenare i vari trasformatori visti in precedenza nella pipeline nello stesso istante.

Codice Alternativo Feature Transformers + Feature Extractors

```

## Feature Transformations
from pyspark.ml.feature import Tokenizer, StopWordsRemover, CountVectorizer, IDF, StringIndexer, OneHotEncoder

has_sentiment_to_num = StringIndexer(inputCol = "airline_sentiment", outputCol = "sentimentIndex")
#data = has_sentiment_to_num.fit(data).transform(data)

onehotencoder_sentiment_vector = OneHotEncoder(inputCol="sentimentIndex", outputCol="sentiment_vec")
#data = onehotencoder_sentiment_vector.fit(data).transform(data)

has_airline_to_num = StringIndexer(inputCol = "airline", outputCol = "airlineIndex")
#data = has_airline_to_num.fit(data).transform(data)

onehotencoder_airline_vector = OneHotEncoder(inputCol="airlineIndex", outputCol="airline_vec")
#data = onehotencoder_airline_vector.fit(data).transform(data)

tokenizer = Tokenizer(inputCol = "text", outputCol = "token_text")
#data = tokenizer.transform(data)

from pyspark.ml.feature import NGram
ngram = NGram(n=2, inputCol="token_text", outputCol="ngrams")
#data = ngram.transform(data)

count_vec = CountVectorizer(inputCol = "ngrams", outputCol = "ngrams_c_vec")
idf = IDF(inputCol = "ngrams_c_vec", outputCol = "ngrams_tf_idf")

```

Codice Pipeline Alternativa

```

from pyspark.ml import Pipeline

data_prep_pipe = Pipeline(stages=[has_sentiment_to_num, has_airline_to_num, onehotencoder_airline_vector,
                                  onehotencoder_sentiment_vector, tokenizer, ngram, count_vec, idf, clean_up])

cleaner = data_prep_pipe.fit(data)

clean_data = cleaner.transform(data)

```

Output n-gram

```

clean_data.select("ngrams").collect()

[Row(ngrams=['@virginamerica what', 'what @dhepburn', '@dhepburn said.']),

```

Codice Modello

```
from pyspark.ml.classification import NaiveBayes

mnb = NaiveBayes(modelType="multinomial", featuresCol = 'features',
                  labelCol = 'sentimentIndex',
                  predictionCol = 'prediction')
```

Il modello di machine learning utilizzato è il multinomial NaiveBayes. labelCol indica la variabile dipendente ovvero sentimentIndex mentre le variabili indipendenti da dare in input al nostro modello di classificazione sono le features indicate al passaggio precedente.(Vector Assembler). prediction è la colonna che contiene le previsioni.

Codice divisione Train e Test Set

```
(training,testing) = clean_data.randomSplit([0.7,0.3], seed = 1)

print("Number of training set rows: %d" % training.count())
print("Number of test set rows: %d" % testing.count())
```

```
Number of training set rows: 7349
Number of test set rows: 3096
```

Codice per estrarre le prime cinque previsioni

```
mnbModel = mnb.fit(training)
mnbpredictions = mnbModel.transform(testing)

mnbpredictions.select("airline_sentiment","sentimentIndex","negativereason","probability","prediction")\
    .orderBy("probability", ascending=False) \
    .show(n = 5, truncate = 30)
```

Output

airline_sentiment	sentimentIndex	negativereason	probability	prediction
negative	0.0	Lost Luggage	[1.0,6.618988623859565E-17,...]	0.0
negative	0.0	Cancelled Flight	[1.0,2.829170972493006E-17,...]	0.0
negative	0.0	Late Flight	[1.0,2.495455989876116E-17,...]	0.0
negative	0.0	Customer Service Issue	[1.0,2.3827410439237054E-17,...]	0.0
negative	0.0	Customer Service Issue	[1.0,1.5039785416418753E-17,...]	0.0

only showing top 5 rows

Codice Matrice di Confusione

```
sentiment_true = mnbpredictions.select(['sentimentIndex']).collect()
sentiment_pred = mnbpredictions.select(['prediction']).collect()

from sklearn.metrics import classification_report, confusion_matrix
print(classification_report(sentiment_true, sentiment_pred))
```

Output

	precision	recall	f1-score	support
0.0	0.97	0.31	0.47	2174
1.0	0.30	0.70	0.42	463
2.0	0.32	0.91	0.47	459
accuracy			0.46	3096
macro avg	0.53	0.64	0.46	3096
weighted avg	0.78	0.46	0.47	3096

La matrice di confusione si riferisce al test set. Nel test set abbiamo in totale 3096 osservazioni. 0 rappresenta la classe dei negativi, 1 rappresenta la classe dei neutrali mentre 2 rappresenta la classe dei positivi. Quindi nel test set abbiamo 2174 osservazioni che rappresentano la classe dei tweet negativi, 463 osservazioni che rappresentano la classe dei tweet neutrali e infine 459 osservazioni che rappresentano la classe dei tweet positivi.

Precision = (TRUE POSITIVE) / (TRUE POSITIVE + FALSE POSITIVE)

Recall = (TRUE POSITIVE) / (TRUE POSITIVE + FALSE NEGATIVE)

Codice Valutazione Modello

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
evaluator = MulticlassClassificationEvaluator(predictionCol="prediction",labelCol = 'sentimentIndex')
print("Accuracy:\t",evaluator.evaluate(mnbpredictions))
print("MisClassificationRate:\t",1 - evaluator.evaluate(mnbpredictions))
```

Output

```
Accuracy:          0.46553530524755954
MisClassificationRate:  0.5344646947524405
```

Correlazione retweet_count -airline_sentiment_confidence

```
data.stat.corr('retweet_count','airline_sentiment_confidence')
```